

คู่มือเรียนจากระบบ VeriForge ที่สร้างจริง

เอกสารนี้เก็บไว้สอนหลังงานพัฒนาเสร็จตามคำสั่งล่าสุด ผู้ใช้ยังไม่ถูกถือว่าเรียนหรือเข้าใจหัวข้อใดแล้วเพียงเพราะมีโค้ดหรือ tests ผ่าน

เริ่มอ่านจากภาพรวม

เปิด dashboard แล้วดู Changes, Releases, Financial lab และ Audit trail สี่หน้าตอบคนละคำถาม: เปลี่ยนอะไร / ปล่อยอะไร / เงินบันทึกถูกหรือไม่ / ใครทำอะไร จากนั้นเปิด [Architecture](ARCHITECTURE.md) เพื่อดูขอบเขตความเชื่อถือ

ลำดับบทเรียนเมื่อพร้อม

1. ****หนึ่ง payment:**** ``api/app.py`` → ``finance.transfer()`` → PostgreSQL → HTTP response
2. ****retry และ race:**** ใช้ key เดิมใน Financial lab และอ่าน ``test_concurrent_retry_posts_once``
3. ****transactions:**** ทดลอง fault ก่อน commit และดู payment/postings/balances ว่าถูกยกเลิกพร้อมกัน
4. ****change proposal:**** ดู source, digest, identity และ provenance ของ change
5. ****tests/scanners:**** อ่าน ``assurance.verify()`` และผลจริงของ 5 mandatory checks
6. ****AI boundaries:**** ดูข้อเสนอที่ถูก block แล้วเทียบกับข้อเสนอที่แก้ผ่าน ห้ามใช้คำตอบของ AI แทนผลทดสอบ
7. ****approval/deploy:**** เปลี่ยนเป็น deployer identity และติดตาม image ID เดียวกันจาก evidence ถึง runtime
8. ****incident/recovery:**** ฉีด process stop แล้วอ่าน metrics, timeline และการกู้คืน
9. ****data/evaluation:**** อ่าน UCI manifest และแยกข้อมูลต้นทางจริงกับ workload ที่แปลงแล้ว
10. ****operations:**** ทดลอง restore ลง DB ใหม่ ดูคำสั่งใน runbook และอธิบายข้อจำกัดของ same-machine backup

ฟังก์ชันหลักและเหตุผล

``finance.minor_units(amount)``

แปลงจำนวนเงินเป็นหน่วยย่อยจำนวนเต็ม รับ string/decimal คำนวณ integer ตรวจ finite, positive, maximum และทศนิยมไม่เกินสองหลักก่อนแปลง ไม่มี I/O จึงไม่มี partial effects หาก validation ล้ม ทดสอบด้วย negative, zero, NaN, infinity และ fraction เล็กเกิน precision

``finance.transfer(tenant, source, destination, amount, currency, key)``

Caller คือ payment API ที่ตรวจ operator role แล้ว Tenant มาจาก identity ไม่รับจาก body ผลลัพธ์คือ payment record และ replay flag มี side effects ที่ accounts/payments/postings/audit ภายใน transaction เดียว

ลำดับ: normalize amount → fingerprint → lock tenant/key → คั้นผลเก่าหรือ reject conflict → lock accounts ตามลำดับคั้งที่ → ขอ quote จาก active release ถ้ามี → ตรวจ quote ด้วย trusted invariant → เขียน payment/3 postings/balances → commit

เหตุผลที่ตรวจซ้ำใน kernel: AI-generated fee code อาจผิด แม้ผ่านชุดทดสอบบางส่วน เงินจึงต้องมี rule ที่เชื่อถือได้อีกชั้น Unique constraint เป็นแนวรับสุดท้ายของ duplicate key การ retry ก่อน commit จะรอ lock; หลัง commit จะได้ผลเดิม ก่อน commit crash จะ rollback ทั้ง transaction

ข้อจำกัด: fees account เป็นจุด contention; THB/USD ใช้ precision สองหลักเท่านั้น; ไม่อ้าง exactly-once ข้าม external bank; compensation ยังไม่อยู่ใน transfer use case นี้

``identity.authenticate()` และ `require()``

อ่าน Bearer token เปรียบเทียบ hash แบบ constant-time กับ per-role tokens ที่สร้างในเครื่อง คั้น Principal หรือ 401 จากนั้น endpoint ระบุ roles ที่อนุญาต ถ้าไม่ผ่านคั้น 403 การซ่อนปุ่ม UI เป็นแค่ UX; server เป็นผู้บังคับจริง

ข้อจำกัด: static local tokens ไม่มี OIDC/session expiry/rotation service ห้ามเผยแพร่เป็น production authentication โดยไม่เพิ่มระบบเหล่านี้

``workflow.create_change()` และ `queue_verification()``

สร้าง immutable proposal พร้อม source digest และ builder identity จากนั้นสร้าง durable job เมื่อ state อนุญาต สถานะอยู่ใน DB ไม่หายเมื่อ restart Audit เก็บการเสนอและการขอตรวจ โดยไม่มี secret ในรายละเอียด

``workflow.work_once()``

Worker claim งานด้วย row lock และ SKIP LOCKED ตั้ง lease/claim token แล้วปล่อย transaction ก่อนรันงานซ้ำ หลังตรวจจะกลับมาเช็ค token อีกครั้งก่อนเขียนผล งานที่ถูก cancel หรือถูก worker ใหม่ reclaim จะไม่ถูก worker เก่าเขียน PASS ทับ หาก process crash lease หมดอายุแล้วทำซ้ำได้

ข้อจำกัด: การตรวจอาจรันซ้ำได้ จึงห้ามให้ verification ไปเขียน production การสร้าง artifact อาจเหลือ cache/container จาก failure ต้องตรวจและจัดการเฉพาะ resource ของงานนั้น

``assurance.inspect_source()` และ `verify()``

AST allowlist จำกัดภาษาเฉพาะสอง pure functions เพื่อเปิดความสามารถอ่านไฟล์/เครือข่าย/เปิด process ก่อน execute Source ต้องผ่าน scans จริงและ acceptance suite ใน Docker ที่ไม่มี network และ credentials ผลคือ evidence bundle พร้อม image ID และ HMAC ไม่ใช่คำตอบว่า “ดูแล้วปลอดภัย”

`decision()` ตรวจสอบ required checks ครบและสถานะจริง FAIL → BLOCK; ERROR/missing → INCONCLUSIVE; ผ่านทั้งหมดจึง PASS ความหมายคือผ่าน policy เวอร์ชันนี้ ไม่ใช่พิสูจน์ว่าไม่มี bug ใดในโลก

`scanners.source\_scans()` และ `image\_scan()`

Gitleaks ตรวจสอบ secrets; Semgrep ใช้ rules ที่ commit ในโปรเจกต์; Trivy ตรวจสอบ packages ใน image ที่จะ deploy ผ่าน image archive Scanner ไม่ได้รับ Docker socket หรือ secrets ของ control plane เก็บ tool image ID/report ไว้ ผลผิดปกติ/เครื่องมือขาดเป็น ERROR ไม่ใช่ไม่มี finding

`ai.build()`, `review()` และ `diagnose()`

Builder ได้ issue/source ที่เกี่ยวข้องและคืน structured proposal Verifier ได้ requirement/source/evidence แบบคนละ context ไม่มี Builder reasoning trace ผล model ต้องผ่าน Pydantic และ evidence refs ต้องตรงกับ check ที่มีจริง Diagnose คืน hypothesis และ uncertainty ไม่อ้างพิสูจน์ root cause หรือรันคำสั่งที่ไม่ได้รับ

โมเดลตัวเดียวกันยังอาจพลาดแบบเดียวกันได้ ความเป็นอิสระที่ทำให้จริงคือ context/authority/trusted tests ไม่ใช่ความเป็นอิสระทางสถิติของ models Budget จำกัด calls และเก็บ usage จริง

`workflow.approve()` และ `check\_release()`

ตรวจสอบสถานะ, proposer != approver, HMAC, source/image digest, policy และ review ต้องตรงกัน Approval ใช้เฉพาะ artifact นี้ ไม่ย้ายไป source ใหม่ได้ หาก trusted runtime/test suite เปลี่ยนต้องตรวจใหม่

`runtime.deploy()`

รับ approved change จาก deployer identity ใช้ image ID สร้าง container จริงแล้ว probe ก่อนเปลี่ยน ACTIVE การเขียนสถานะและ side effects ข้าม Docker/DB ไม่ใช่ transaction เดียว จึงใช้ชื่อ release คงที่เพื่อให้ retry ตรวจสอบพบ container เดิมและไม่สร้างซ้ำ

`runtime.measure()` / `observe()` / `monitor.tick()`

ยิงคำขอสองครั้งต่อ key เก็บเวลา/error/duplicate แล้วเปรียบเทียบ policy Runtime monitor ทำซ้ำตามช่วงเวลา หากผิดให้สร้าง incident และ recover ไป standby หรือ restart approved image ก่อนวัดซ้ำ การหยุด container ทดสอบเกิดเฉพาะ target ที่บันทึกใน deployment record

ค่า p95 ในชุดนี้คือเวลาของ replay scenario สอง requests ไม่ใช่ single-request latency และไม่ใช่ user traffic จริง ต้องอ่าน provenance/sample count ด้วย

`scripts/backup.py` และ `evaluate\_data.py`

Backup ใช้ pg_dump แล้ว pg_restore ลง DB ชั่วคราวคนละชื่อ ไม่ทับ live DB ส่วน dataset script โหลดข้อมูล UCI จริง บันทึก checksum/license/transform และ replay ผ่าน financial kernel การ map GBP numerical amount ไป THB sandbox units ไม่ใช่อัตราแลกเปลี่ยนจริงและมีป้ายกำกับชัด

สิ่งที่ให้คุณทดลองเองเมื่อเริ่มสอน

แต่ละรอบเลือกเพียงหนึ่งเรื่อง: เปลี่ยน amount แต่ใช้ key เดิม, ทำ scanner ให้ unavailable, ลอง approve ด้วย builder token, เปลี่ยน artifact หลังตรวจ, stop target, restore backup จากนั้นทายผล → รัน → อ่าน evidence → อธิบายว่าจุดไหนบังคับกฎ ไม่ต้องพิมพ์ implementation ตามทั้งหมด

วิธีวัดความเข้าใจ

คุณควรอธิบาย input/output, จุดเปลี่ยนข้อมูล, ผู้มีสิทธิ์, failure boundary และ test ที่พิสูจน์กฎสำคัญได้ การจำ syntax ทุกบรรทัดไม่ใช่เป้าหมาย ใช้ [roadmap](LEARNING-ROADMAP.md) เมื่อต้องการเจาะหัวข้อเพิ่ม

VeriForge local v1 — Architecture and trust boundaries

Product boundary

VeriForge accepts a change to a bounded Python financial-policy module (`fee\_minor`, `should\_post`), executes independent checks, obtains an independent-context model review, requires a different approver identity, and deploys the verified Docker image to a local target. PostgreSQL stores the durable ledger, changes, evidence, jobs, releases, usage and audit events.

This is a functioning local assurance platform for one tenant and one target type. Arbitrary repository execution, enterprise OIDC, multi-tenant cloud hosting and real payment-provider settlement are not enabled by this local configuration. Adding these requires new trust boundaries, not simply removing source restrictions.

Components

```
flowchart TD
    UI[React console] --> API[FastAPI + bearer identities]
    API --> DB[(PostgreSQL)]
    API --> B[Builder context / Ollama]
    B --> C[Immutable change proposal]
    C --> Q[Durable verification job]
    Q --> W[Worker]
    W --> S[AST + Gitleaks + Semgrep + Trivy]
    W --> T[Networkless non-root Docker acceptance tests]
    S --> E[HMAC evidence bound to source + image + suite]
    T --> E
    E --> V[Independent Verifier context / Ollama]
    V --> P[Deterministic policy]
    P --> A[Operator approval]
    A --> D[Deployer identity]
    D --> R[Approved Docker image + HTTP probes]
    R --> M[Periodic runtime monitor]
    M --> I[Incident + rollback or restart]
    API --> F[Financial kernel]
    F --> R
    F --> DB
```

Source and artifact identity

Changes have a SHA-256 of the submitted source. Builder can supply only the two-function module, not tests, server, Dockerfile or policy. The capability allowlist forbids imports, attributes, module-level execution and arbitrary function calls. Docker tests run without network, root, secrets, host sockets, writable image or unbounded resources. These restrictions make the supported language deliberately narrow.

The build uses a pinned Python base digest. Evidence records actual scanner image IDs, source digest, test-harness/runtime hashes and final image ID. Missing mandatory reports are INCONCLUSIVE. Findings cause BLOCK. AI cannot override either. Approval must match the still-current artifact, source and policy and expires after 24 hours. The deployer executes the image ID, not a mutable tag. Re-verification is required after changes to the trusted harness/runtime.

HMAC detects altered evidence for actors without the signing key. It is not a public attestation, append-only external log, or protection against the machine administrator/database superuser. Local API, worker and deployer run under the owner's OS account; production separation would use separate service accounts and hosts.

Money correctness

The financial kernel stores integer minor units. Tenant/key advisory locking and a unique tenant/key constraint serialize replays. The request fingerprint rejects changed parameters. Accounts are locked in stable order. A single DB transaction writes payment, debit, credit, fee and balance changes. A deferred DB constraint checks three postings and zero net delta before commit. Posted financial records reject update/delete.

When a release is active, the kernel requests its fee quote and validates the returned fee independently against the trusted pricing contract. The image ID becomes the pricing version. If the release is unavailable or wrong, a new payment fails closed. Existing replay results can still be returned without posting again. Before the first release, the explicit bootstrap fee rule is used.

The target's in-memory replay ledger is a probe fixture, not the durable financial record. Actual financial records live in PostgreSQL. Process rollback does not reverse committed payments. Compensation/refunds beyond the current transfer use case are future work.

State and concurrency

``PROPOSED → QUEUED → VERIFYING → AWAITING_APPROVAL → APPROVED → DEPLOYED``

Other terminal/review states: ``BLOCKED``, ``INCONCLUSIVE``, ``NEEDS_HUMAN``. Jobs use ``FOR_UPDATE SKIP LOCKED``, bounded attempts, leases and claim tokens. A cancelled or superseded claim

cannot write successful evidence. Expired leases are reclaimable; execution itself may run again, so verification is side-effect-free relative to the ledger. Docker deployment uses a stable name derived from the change ID to recover interrupted launches. Only one release operation is applied at a time via an advisory lock.

Identity matrix

| Identity | Read | Propose | Verify | Approve | Deploy | Financial writes | Fault injection |

| --- | --- | --- | --- | --- | --- | --- | --- |

| viewer | yes | no | no | no | no | no | no |

| builder | yes | yes | request | no | no | no | no |

| verifier | yes | no | request | no | no | no | no |

| operator | yes | request AI/fixture | request | yes | no | yes | local target only |

| deployer | yes | no | no | no | yes | no | no |

AI contexts never receive tokens, DB credentials, signing keys or Docker control. Same installed model is used for cost reasons; separate contexts and constrained authority are implemented, provider diversity is not claimed.

Data and measurement

`artifacts/dataset-evaluation.json` records UCI Online Retail provenance, SHA-256, row selection and amount transformation. It contains real retail-derived amounts replayed as synthetic THB test funds without FX conversion. It is not bank traffic or a fraud-labelled assurance benchmark.

HTTP metrics come from actual requests, Prometheus histograms and OpenTelemetry spans saved locally. Runtime p95 measures a two-request replay scenario, not a single request. Periodic observations label synthetic traffic and record sample count/time. Recovery is based on measured failures, duplicate postings or latency threshold; AI incident analysis remains a hypothesis and cannot execute recovery.

ADR-001 — Small control plane

Use FastAPI + PostgreSQL jobs and React; postpone Redis/Kafka/Kubernetes. This reduces cost and operational dependencies while retaining durable state. The local shared fees account limits throughput; measure before distributing it.

ADR-002 — Bounded changes first

Accept only financial-policy Python with trusted tests and a restricted capability set. This produces an auditable working software-change loop without pretending a single Docker container securely runs arbitrary hostile repositories. Extend with VM-level isolation and separate execution hosts before allowing arbitrary builds.

ADR-003 — Local credentials and AI

Use generated per-role bearer tokens and installed Ollama. This avoids external spend and establishes role enforcement, but is not enterprise identity lifecycle management. Rotate by editing generated secrets locally then restarting services; do not commit them. Stronger identity is required before public hosting.

Runbook — local operation

Start

Prerequisites: Python 3.13, uv, Node 22, Docker Desktop running, Ollama with `qwen3.5:9b`. First dependency/image/dataset downloads need network access; model calls are local. Keep at least 3 GB free for scanners/builds.

From the project root:

```
./run-local.sh
```

Open `http://127.0.0.1:8123`. Local per-role tokens are in `.local/config.json`; open this file privately and paste the appropriate token. Never share or commit it. Browser credentials remain in sessionStorage until sign out/tab session end. Operator approves and controls local lab experiments; deployer is a different token.

The start script creates credentials once, synchronizes the locked dependencies, starts PostgreSQL, builds the UI if absent, applies migrations, and runs the API, worker and runtime monitor. For changed frontend source run `cd frontend && npm ci && npm run build`, then restart the API. No public tunnel is configured.

Main journey

1. Sign in as operator; open Changes and load a failing fixture or ask local AI for a proposal.
2. Open the change and select Verify. Worker performs real source/image scans and Docker acceptance tests.
3. A failure is BLOCKED; errors/missing reports are INCONCLUSIVE. Inspect the exact check before retrying. An AI repair is a new change, never an in-place replacement of approved source.
4. After all gates and AI review pass, operator approves the evidence-bound artifact.
5. Sign out and sign in as deployer; deploy the approved change. Local HTTP probes must pass before activation.
6. Sign in as operator; submit and replay a financial payment. PostgreSQL deduplicates it and checks the candidate quote against the trusted rule.
7. On Releases, observe traffic or inject a local process stop. The system records an incident and verifies rollback/restart. This uses test funds only.

Verification commands

```
.venv/bin/ruff check src tests scripts
.venv/bin/pytest -q --junitxml=artifacts/tests.xml
env PYTHONPATH=src .venv/bin/python scripts/verify_lifecycle.py
env PYTHONPATH=src .venv/bin/python scripts/evaluate_data.py --limit 100
env PYTHONPATH=src .venv/bin/python scripts/backup.py --verify-restore
cd frontend && npm run build && node tests/browser.mjs
```

The lifecycle test calls a real model. A rejected patch is a valid safety outcome but not a successful end-to-end release run. Do not edit trusted tests to make a generated patch pass. Repair receives the failed evidence and is capped at two automated attempts in the lifecycle script.

Incident handling

| Symptom | Check | Action |

| --- | --- | --- |

| UI cannot load API | API logs / health | Restart API; preserve database volume |

| Job queued | Worker process | Start worker; do not approve queued changes |

| VERIFYING stale | jobs lease/attempts | Expired lease is reclaimed; after 3 attempts state is INCONCLUSIVE |

| Scanner ERROR | Evidence detail + disk + scanner image/DB | Repair availability, then re-verify; never map error to pass |

| Model unavailable | Ollama service/model + daily call cap | Restore service or wait/reset configured budget deliberately; no silent fallback |

| Financial payment 503 | Active release health / quote | Recover release; existing idempotent result remains retrievable |

| Runtime regression | Release observations/incident timeline | Monitor rolls back to standby or restarts approved image; verify recovery metrics |

| Data loss concern | Last dump and checksum | Restore to a NEW database first; never overwrite live DB automatically |

Backups and recovery limits

`scripts/backup.py --verify-restore` writes a custom PostgreSQL dump under `.local/backups/`, restores into a uniquely named validation DB, checks readable data and removes only that temporary

DB. It never drops the application DB. Backups contain business/evidence data; keep permissions restricted.

Current backups are on the same machine. For machine-loss recovery, copy the dump and signing-key/credential configuration into an encrypted off-machine location that you control. No off-machine account is configured. Set backup frequency and retention based on your chosen RPO; no RPO/RTO or availability SLA has been measured/promised.

DB backup does not contain Docker images. Preserve or rebuild approved images and re-verify before releasing if artifact IDs differ. Archive traces separately if needed; their current local file is ``.local/traces.jsonl``.

Controls and bounded costs

- ``VF_MODEL_DAILY_CALLS`` defaults to 30; calls and actual tokens are recorded in PostgreSQL.
- ``VF_BUILDER_MODEL`` and ``VF_VERIFIER_MODEL`` default to installed ``qwen3.5:9b``; separate prompts but same weights.
- ``VF_OLLAMA_URL`` defaults to `localhost:11434`. No cloud API spend is configured.
- ``VF_DATABASE_URL`` optionally overrides the generated local DB connection.
- ``VF_GITHUB_REPOS`` explicitly allowlists read-only imports such as ``owner/repo``; the supported PR changes only ``policy.py``. No GitHub write/merge token is used.
- Prometheus endpoint ``/api/metrics`` requires bearer auth. OpenTelemetry spans are written locally, not sent to a third party.

Stop and preserve data

Ctrl+C the start script to stop API/worker/monitor. ``docker compose --env-file .local/compose.env stop db`` stops only this project's database. Release containers remain separately manageable by their recorded IDs. Never use broad Docker pruning or ``down -v`` as normal shutdown.

Before public or real-money use

This release is a local pilot, not a public multi-tenant financial service. Required upgrades include enterprise identity/token lifecycle, separate OS/service identities and execution hosts, HTTPS, rate limits and abuse controls, external tamper-resistant evidence, off-machine recovery, appropriate compliance review, and actual settlement/reconciliation integration. These are explicit boundaries, not features implied by a successful local run.

Validation record — 21 September 2026

This record separates observed execution from implementation and future work. Local machine: macOS ARM64, Python 3.13.9, PostgreSQL 17, Docker Desktop, Ollama qwen3.5:9b. No cloud billing or real-money settlement was configured.

Verified evidence

| Check | Observed result | Evidence |

| --- | --- | --- |

| Backend unit/integration | 30 passed, 2 dependency deprecation warnings | artifacts/tests.xml |

| PostgreSQL concurrency | 24 simultaneous same-key requests created one payment | tests/test_finance_integration.py |

| Rollback/validation | pre-commit failure leaves no payment; invalid money rejected | backend suite |

| Authorization/evidence | invalid roles, self approval, tampered evidence/source rejected | backend suite |

| Security execution | baseline passed real Gitleaks, scoped Semgrep and Trivy | changes.evidence in PostgreSQL |

| Public-data replay | 100 payments, 0 failures, 0 imbalanced ledgers; two requests per row | artifacts/dataset-evaluation.json |

| Replay p95 | 52.96 ms per TWO-request scenario in recorded run | same report; sequential local workload |

| Frontend | final production build and desktop/mobile journey passed | artifacts/*.png |

| AI change lifecycle | actual AI repair passed all gates, deployed, then injected fault recovered by rollback | artifacts/lifecycle.json completed PASS |

| Backup | pg_dump restored into a new temporary database; 0 ledger imbalances | .local/backups/*.json |

Full model → repair → release → recovery acceptance is recorded incrementally in artifacts/lifecycle.json. Only a final completed/PASS step is a successful lifecycle run; earlier failed runs remain in database/audit. The latest status is reconciled in PROGRESS.md after execution.

Failures discovered, not hidden

- A generated patch accepted float amounts. Trusted financial tests blocked it.
- A repair used a forbidden `isinstance()` call. Capability policy blocked it. Repair context now includes all failed checks, not only financial assertions.
- An early Gitleaks stdout report was empty. Adapter now reads the scanner's report file; empty/malformed output cannot pass.
- Trivy found HIGH vulnerabilities in unused pip-vendored packages in the target base. The standard-library-only runtime now removes the unused installer packages; required scans then passed.
- A Docker build timed out. The change was INCONCLUSIVE, not released. Retrying after unloading the local model completed the build.
- Docker's OCI index identifier differed from the platform config identifier reported by Trivy. The adapter binds both through the exact Docker-exported archive. See [OCI image manifest specification](<https://specs.opencontainers.org/image-spec/manifest/>).
- Machine free disk fell below 0.5 GB during the run. Do not treat this workstation as a reliable production host; leave several GB headroom before further model/image downloads.

Data source and interpretation

[UCI Online Retail](<https://archive.ics.uci.edu/dataset/352/online+retail>), DOI 10.24432/C5BW33, CC BY 4.0. Source archive SHA-256 and retrieval time are recorded in the evaluation artifact. First 100 eligible positive non-cancelled line-item amounts are used; quantities × GBP unit price are mapped numerically into THB sandbox units WITHOUT exchange-rate conversion. No historical bank account reconstruction or fraud labels are claimed.

Source provenance: PUBLIC_BENCHMARK. Workload execution: SYNTHETIC_REPLAY. Real operational evidence: actual tests/scanners/model requests/deployments. Process-stop exercise: INJECTED_FAULT.

Not demonstrated by these results

- No enterprise production deployment, SLA, real customer pilot or actual payment-provider settlement.
- No representative held-out bug benchmark, confidence interval, verifier recall/false-block generalization, distributed load or 100-agent scalability.
- Same local model for both roles: separate context/authority, not independent error distributions.

- Read-only GitHub PR adapter implemented and allowlist rejection tested; no authorized external repository configured and no hosted CI run.
- No OIDC/ABAC service, distributed identity isolation, cloud IaC, cross-service tracing stack, multi-host HA or off-machine restore.
- Local staged probes are not a traffic-split canary against live user traffic.
- Fixed trusted adversarial cases exist; the Verifier does not autonomously add executable test code.
- Incident storage and AI hypothesis adapter exist; cross-incident semantic retrieval and automated postmortem learning are not implemented.
- HMAC and local audit are not external tamper-proof attestations against the workstation administrator.
- Two dependency deprecation warnings remain in the TestClient stack; tests pass.

These boundaries preserve the larger roadmap rather than pretending all ten enterprise phases have been completed.

แผนเรียนรู้ทั้งระบบจากโค้ดที่ทำงานจริง

ทุก module ใช้กระบวนการใน [ข้อตกลงการทำงาน](WORKING-AND-LEARNING-AGREEMENT.md) และบันทึกสถานะใน [PROGRESS](PROGRESS.md) ลำดับนี้เป็นเส้นทางเริ่มต้น ปรับได้ตาม prerequisite และผลการทดลอง ไม่ใช่ตารางเวลารับประกัน

ภาพรวมระบบที่ต้องอธิบายได้เมื่อจบ

```
Requirement / Issue / Incident
→ Builder: inspect → propose patch
→ Identity + repository/policy boundary
→ Isolated execution: tests / scans / adversarial cases
→ Evidence bundle + Verifier findings
→ Deterministic decision
→ Human approval (ตาม policy)
→ Deploy approved artifact
→ Runtime observation
→ Detect incident → diagnose → stop/rollback/patch
→ Verify again → postmortem → regression memory
```

M00 — พื้นฐานเครื่องมือและแผนที่โปรเจกต์

- **หลักการ:** file/folder, process, terminal, dependency, virtual environment, Git diff, environment variable และ secret
- **ลงมือ:** ตรวจสอบเครื่องและสถานะ repository; เตรียมคำสั่งรันมาตรฐานและ project layout ที่เล็กที่สุด ยังไม่สมัคร cloud
- **พาด่าน:** entry point, configuration และเส้นทาง request โดยระบุอะไรยังไม่ implement
- **ทดลอง:** configuration ที่จำเป็นหายแล้วระบบควรแจ้งอะไร โดยไม่แสดง secrets
- **ตรวจรับ:** documented startup ใช้ได้จริงบนเครื่องนี้ ระบุ prerequisite ที่ยังขาด
- **Teach-back:** code file กับ process ต่างกันอย่างไร? dependency อยู่ตรงไหน? ทำไมไม่ commit secrets?

M01 — API และข้อมูลหนึ่งรายการ

- **หลักการ:** HTTP method/status, JSON, function, type, validation, exception, input/output

- ****ลงมือ:**** payment request ด้วย amount/currency และ identifiers; health endpoint; แยก API schema จาก domain logic
- ****พาดู:**** route → validation → domain function → response พร้อมตัวอย่าง valid/invalid input
- ****ทดลอง:**** negative amount, missing field, malformed input
- ****ตรวจรับ:**** contract ชัดเจนและ tests พิสูจน์ invalid input ไม่สร้าง financial effects
- ****Teach-back:**** อะไรควรตรวจที่ API และอะไรต้องตรวจใน domain? HTTP success พิสูจน์ว่าเงินถูกหรือไม่?

M02 — Database, transaction และ ledger

- ****หลักการ:**** table/key/index, constraint, commit/rollback, migration, monetary precision และ debit/credit
- ****ลงมือ:**** PostgreSQL สำหรับ account/payment/ledger; deterministic fee rule; transaction boundary
- ****พาดู:**** read/write path และจุดที่ posting ต้องสำเร็จพร้อมกัน
- ****ทดลอง:**** ล้มก่อน commit แล้วเทียบจำนวน records/balances; จำนวนเงินที่มี rounding edge
- ****ตรวจรับ:**** actual PostgreSQL integration tests; ledger สมดุลแยก currency; ไม่ทิ้ง partial posting
- ****Teach-back:**** DB rollback ต่างจาก release rollback อย่างไร? float ทำให้ยอดผิดได้อย่างไร?

M03 — Idempotency, retries และ concurrency

- ****หลักการ:**** request identity, payload fingerprint, race condition, isolation และ uniqueness
- ****ลงมือ:**** ทำ duplicate-free reproducer ใน test-only fixture ก่อนสร้างทางแก้; scope key ตาม tenant/operation และกำหนด key lifecycle
- ****พาดู:**** รับ key → ตรวจ payload → จอง/อ่าน operation → financial transaction → เก็บผล → ตอบ client
- ****ทดลอง:**** timeout หลัง commit, concurrent same-key calls, same-key different amount, restart/retry
- ****ตรวจรับ:**** ไม่สร้าง financial effect ซ้ำ; payload mismatch ปฏิเสธ; tests รันบน DB จริงและไม่พึ่ง sleep อย่างเดียวเพื่อสร้าง race
- ****Teach-back:**** ทำไม `if status != completed` ไม่พอ? อะไรรับประกัน uniqueness? key หมดอายุมีผลอย่างไร?

M04 — Tests, trusted controls และ evidence

- ****หลักการ:**** unit/integration/regression/property-based tests, test oracle, exit code, raw artifact, evidence provenance
- ****ลงมือ:**** trusted regression suite, runner output parser และ evidence bundle ผูกกับ revision/configuration
- ****พาด่าน:**** test input → execution → assertion → raw output → normalized evidence
- ****ทดลอง:**** test fail, process timeout, malformed output, missing evidence และ evidence ของคนละ revision
- ****ตรวจรับ:**** แยก PASS/FAIL/ERROR/INCONCLUSIVE ตาม contract; ไม่มี false pass เมื่อ parser/scanner ล้ม
- ****Teach-back:**** tests ผ่านยังไม่พิสูจน์อะไร? ใครแก้ trusted test ได้? screenshot เพียงอย่างเดียวพอหรือไม่?

M05 — Identity และ authorization

- ****หลักการ:**** authentication vs authorization, user/service identity, RBAC, least privilege, segregation of duties
- ****ลงมือ:**** role/permission matrix และ enforcement ตั้งแต่เริ่ม control API; ขยาย integration กับ identity provider ตามความจำเป็น
- ****พาด่าน:**** identity → resource/action → policy → allow/deny; tenant scope ของ evidence และ approvals
- ****ทดลอง:**** Builder approve/deploy ตัวเอง, unauthorized evidence read, cross-tenant request, expired credential
- ****ตรวจรับ:**** negative authorization tests; สิทธิ์ตรวจฝั่ง server ทุก transition ไม่พึ่งซ่อนปุ่ม
- ****Teach-back:**** token ถูกต้องแต่ไม่มีสิทธิ์ได้หรือไม่? role เดียวกันต่าง tenant อ่านข้อมูลกันได้ไหม?

M06 — Durable jobs และ state machine

- ****หลักการ:**** state/transition, queue, lease, heartbeat, retry/backoff, cancellation, at-least-once processing
- ****ลงมือ:**** persistent changes/jobs/events และ worker; transition rules อยู่ฝั่ง server
- ****พาด่าน:**** enqueue → claim → execute → persist → completion; จุดที่ต้อง deduplicate side effects

- ****ทดลอง:**** worker ตายหลังทำงานแต่ก่อน ack, duplicate webhook/job, expired lease, cancel ระหว่างรัน
- ****ตรวจรับ:**** restart แล้วเดินต่อได้; ไม่มี deploy ช้า; invalid transition ถูกปฏิเสธ
- ****Teach-back:**** retry job เท่ากับปลอดภัยเสมอหรือไม่? อะไรต้อง idempotent?

M07 — Security scans และ isolated execution

- ****หลักการ:**** untrusted code, filesystem/network boundary, process/container/VM, secrets, dependency risk
- ****ลงมือ:**** scanner adapters สำหรับ Gitleaks/Semgrep/Trivy ตาม scope; runner แยกจาก credentials/control plane
- ****พาด่าน:**** execution specification → limits → raw report → finding normalization → evidence
- ****ทดลอง:**** scanner unavailable, crafted findings, forbidden network/tool action ใน sandbox
- ****ตรวจรับ:**** ไม่ปล่อยเมื่อ mandatory scan ไม่ครบ; ไม่มี secrets ใน logs; จำกัด resource/time และตรวจสอบเขต runner
- ****Teach-back:**** container อย่างเดียวรับประกัน isolation แค่นั้น? scanner ไม่พบอะไรเท่ากับปลอดภัยหรือไม่?

M08 — Deterministic policy และ risk decision

- ****หลักการ:**** hard gates, soft findings, policy version, exception expiry, explainability, TOCTOU
- ****ลงมือ:**** input schema ของ evidence และ deterministic decision engine ก่อนต่อ LLM
- ****พาด่าน:**** verify evidence identity/completeness → hard gates → risk/review → approval requirement
- ****ทดลอง:**** ต่ำคะแนนแต่ invariant fail, policy เปลี่ยน, exception หมดอายุ, stale evidence
- ****ตรวจรับ:**** same versioned input ให้ decision เดิม; block conditions เอาชนะ score; reason อ้าง evidence จริง
- ****Teach-back:**** risk score มีหน้าที่อะไร? approval เดิมใช้กับ code ใหม่ได้ไหม?

M09 — Builder และ Independent Verifier AI

- ****หลักการ:**** probabilistic model, structured output, tool permissions, context separation, prompt injection, cost budget

- ****ลงมือ:**** provider adapter; Builder เสนอ patch; Verifier รับ requirement/diff/evidence และเสนอ findings/adversarial tests
- ****พาด่าน:**** context assembly → model call → schema validation → constrained tool execution → evidence-linked findings
- ****ทดลอง:**** hallucinated evidence reference, invalid JSON, injected instruction ใน diff/log, provider timeout, budget exhausted
- ****ตรวจรับ:**** ไม่มี silent pass; AI ไม่มีอำนาจลบ hard gate; trusted tests แก่โดย Builder ไม่ได้; model usage บันทึกรจริง
- ****Teach-back:**** คนละ model ทำให้อิสระจริงหรือไม่? Verifier เห็นผลทดสอบแต่เชื่ออะไรได้บ้าง?

M10 — CI/CD, artifacts และ approval

- ****หลักการ:**** commit vs tag vs digest, reproducible inputs, build provenance, supply chain, release identity
- ****ลงมือ:**** CI รัน checks จริง; build artifact; approval ผูก digest; restricted deployer
- ****พาด่าน:**** PR → build/checks → evidence → approval → artifact verification → staging
- ****ทดลอง:**** แก่ source หลังอนุมัติ, สลับ artifact, replay approval, untrusted workflow request
- ****ตรวจรับ:**** deploy ได้เฉพาะ artifact ที่ผ่านและอนุมัติ; identity แต่ละส่วนทำเกินหน้าที่ไม่ได้
- ****Teach-back:**** ทำไมชื่อ image tag อย่างเดียวไม่พอ? ใครเชื่อถือใครใน build chain?

M11 — Runtime, canary และ observability

- ****หลักการ:**** logs/metrics/traces, correlation ID, counter/histogram, SLI/SLO, sampling, baseline, sample size
- ****ลงมือ:**** instrumentation ของ target/control plane; canary หรือ staged rollout ที่ระบุข้อจำกัด
- ****พาด่าน:**** request → span/events → metrics → query → release decision
- ****ทดลอง:**** error rate/latency regression, telemetry missing, traffic ต่ำและ window ไม่พอ
- ****ตรวจรับ:**** dashboard อ้าง metric จริง; ไม่มีข้อมูลต้องไม่เป็น healthy โดยอัตโนมัติ; low sample เป็น inconclusive
- ****Teach-back:**** p95 ต่างจากค่าเฉลี่ยอย่างไร? correlation บอก root cause ได้หรือยัง?

M12 — Incident, recovery และ backup

- ****หลักการ:**** symptom vs cause, blast radius, rollback vs compensation, RPO/RTO, recovery verification
- ****ลงมือ:**** incident timeline/runbook, controlled faults, image rollback, DB restore และ financial reconciliation
- ****พาด่าน:**** detect → contain → diagnose → choose recovery → execute → verify → postmortem
- ****ทดลอง:**** DB lock, process crash, duplicate event และ restore ลง environment ใหม่
- ****ตรวจรับ:**** evidence ของ detection/recovery; schema compatibility; ไม่อ้าง rollback ว่าย้อน external payment; restore ใช้ได้จริง
- ****Teach-back:**** อะไรต้องหยุดก่อนแก้? backup ที่ไม่เคย restore เชื่อถือได้แค่ไหน?

M13 — Datasets, evaluation และ load tests

- ****หลักการ:**** ground truth, sampling, leakage, benchmark contamination, confusion matrix, cost/performance trade-off
- ****ลงมือ:**** dataset manifests, PaySim adapter ตามความเหมาะสม, SWE-bench subset แยกจาก finance benchmark; workload ของระบบเรา
- ****พาด่าน:**** source → transform → replay → raw evidence → metric computation → report
- ****ทดลอง:**** compare deterministic-only / plus Verifier / full Builder loop; repeat with pinned configuration
- ****ตรวจรับ:**** แยก good/bad/unknown; รายงาน false blocks, missed defects, inconclusive, sample size และข้อจำกัด
- ****Teach-back:**** fraud label ใช้เป็น label ของ code bug ได้ไหม? sample น้อยอ้าง 99% reliability ได้ไหม?

M14 — Dashboard, pilot และ maintenance

- ****หลักการ:**** status semantics, evidence drill-down, stale data, operator UX, support/incident ownership, cost controls
- ****ลงมือ:**** dashboard อ่าน API จริง; pilot สำหรับผู้ใช้/repos ที่อนุญาต; retention, backup schedule และ upgrade procedure
- ****พาด่าน:**** UI action → auth → backend transition → event → UI refresh; ทุก metric ย้อนหา source ได้

- ****ทดลอง:**** stale evidence, unauthorized UI action, service restart, exhausted quota และ dependency upgrade
- ****ตรวจรับ:**** end-to-end acceptance suite, operating runbook, known limitations และ budget ที่บังคับจริง
- ****Teach-back:**** ถ้าระบบล่มใครรู้และทำอะไร? metric ไหนเป็นจริง/จำลอง? ขยายระบบเมื่อมีหลักฐานอะไร?

ความสัมพันธ์และการสอนซ้ำ

M05 identity ต้องเริ่มพร้อม control API ไม่ใช่รอจนระบบสมบูรณ์ M13 เก็บ manifest/evaluation foundations ตั้งแต่เริ่มใช้ข้อมูล M14 ทำ UI ที่ละส่วนเมื่อ backend มีข้อมูลจริง ไม่รอท้ายสุด ส่วน M00–M04 เป็นวงจรแรกที่ต้องทำให้รันได้

กลับไปทบทวน module เดิมเมื่อเกิด bug ที่เกี่ยวข้องหรือผู้ใช้อยู่ยังอธิบายไม่ได้ โดยใช้ code/evidence ปัจจุบัน ไม่เริ่มหลักสูตรใหม่ทั้งหมด